OFICINA

ESCOLA PROFISSIONAL

Relatório Breve da Organização do Programa por Funções
10.º Ano | Programação e Sistemas de Informação (PSI) | Python

1. Introdução

O programa desenvolvido consiste num sistema de **Gestão de Stock** para materiais escolares, implementado em Python. A versão entregue representa uma evolução significativa relativamente ao código original, demonstrando como as funções permitem não apenas organizar o código, mas também facilitar a implementação de melhorias estruturais e validações robustas.

2. Como as Funções Organizam o Programa

2.1 Modularidade e Separação de Responsabilidades

O programa está dividido em **seis funções principais**, cada uma com uma responsabilidade única e bem definida:

adicionar_material(stock) – Regista novos materiais com validação de dados (impede quantidades negativas e verifica duplicados)

consultar_stock(stock) – Permite pesquisar o stock de um material específico

atualizar_stock(stock) – Gerencia a adição ou remoção de quantidades, com controlo de erros (ex: impede remover mais do que existe em stock)

exibir_stock(stock) – Apresenta uma tabela formatada com o estado geral de todos os materiais

file_export(stock) – **Melhoria adicionada:** Exporta os dados para um ficheiro .txt com data/hora no nome, permitindo backup e análise posterior

main() – Função principal que coordena o fluxo do programa, apresentando o menu e direcionando para as funções adequadas conforme a escolha do utilizador

2.2 Benefícios da Organização por Funções

Reutilização de Código: A estrutura stock (dicionário) é passada como argumento entre funções, evitando variáveis globais e permitindo que cada função opere sobre os mesmos dados de forma organizada.

Facilidade de Manutenção: Como demonstrado nas capturas de ecrã, quando se adicionou a funcionalidade de exportação (file_export), foi necessário apenas criar uma nova função independente e adicionar uma opção no menu principal, sem alterar as restantes funções existentes.

Legibilidade: O código torna-se autoexplicativo. O nome de cada função indica claramente a sua função (ex: consultar_stock consulta stock), facilitando a compreensão para alunos e professores.

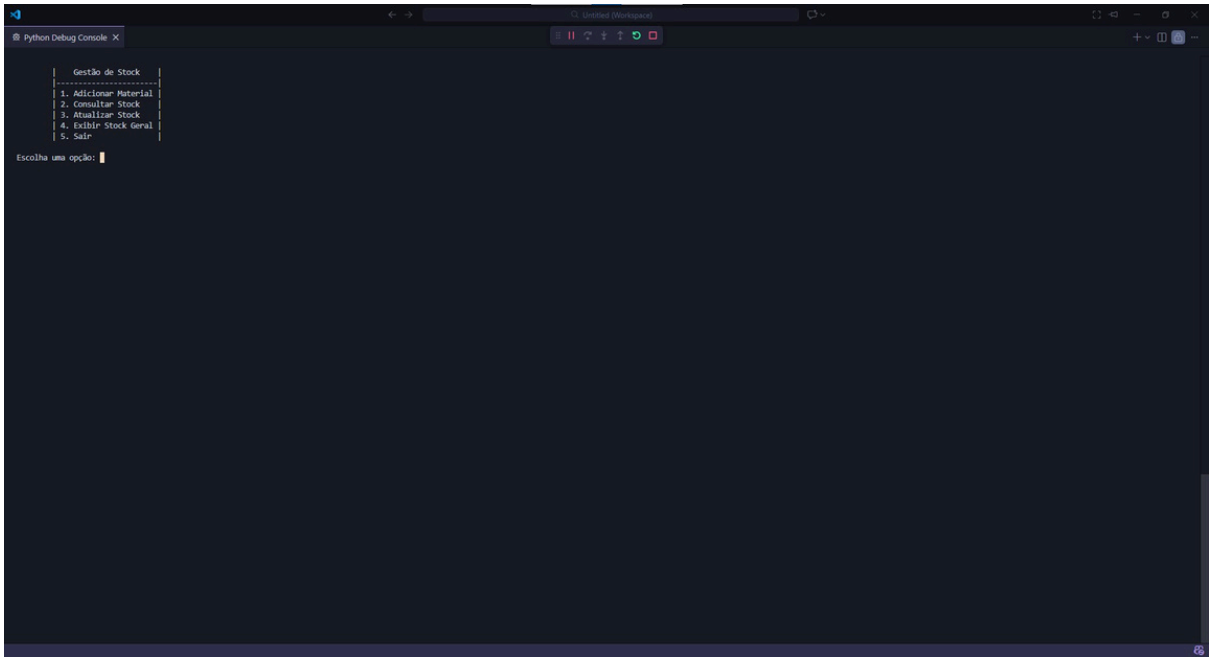
Testabilidade: Cada função pode ser testada individualmente. Por exemplo, na captura de ecrã 02.03, verifica-se que a função atualizar_stock rejeita valores negativos, demonstrando que as validações funcionam corretamente de forma isolada.

3. Evidências de Teste e Validação (Python Debug Console)

As fotografias da **Python Debug Console** documentam a **evolução cronológica e funcional do projeto**, apresentando duas fases distintas de testes que evidenciam o processo de refinamento das funções.

3.1 Fase 1: Versão Original (5 opções)

Python Debug Console 01.01-05



```
Python Debug Console

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 1
Nome do material: ferro
Quantidade inicial de Ferro: 100
Ferro adicionado com sucesso!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
------------------------------
Ferro           100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 2
Nome do material para consulta: ferro
O stock atual de Ferro é: 100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção:
```

```
Python Debug Console

Escolha uma opção: 2
Nome do material para consulta: ferro
O stock atual de Ferro é: 100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 1
Nome do material: ouro
Quantidade inicial de Ouro: -10
Ouro adicionado com sucesso!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
------------------------------
Ferro           100
Ouro            -10

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 3
Nome do material a atualizar: ouro
Deseja adicionar (A) ou remover (R)? a
Quantidade: 10
10 unidade(s) adicionada(s) ao stock de Ouro.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |
```

```
Python Debug Console

Escolha uma opção: 3
Nome do material a atualizar: ouro
Deseja adicionar (A) ou remover (R)? a
Quantidade: 10
10 unidade(s) adicionada(s) ao stock de Ouro.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
------------------------------
Ferro           100
Ouro            0

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 2
Nome do material para consulta: alien
Alien não encontrado no stock.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 3
Nome do material a atualizar: icognita
Icognita não encontrado no stock.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |
```

```
Python Debug Console

Deseja adicionar (A) ou remover (R)? a
Quantidade: 10
10 unidade(s) adicionada(s) ao stock de Ouro.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
------------------------------
Ferro           100
Ouro            0

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 2
Nome do material para consulta: alien
Alien não encontrado no stock.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 3
Nome do material a atualizar: icognita
Icognita não encontrado no stock.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Sair               |

Escolha uma opção: 5
Encerrando o programa...
PS K:\Anunali>
```

Estas capturas demonstram o programa base antes das melhorias estruturais, focando na navegação fundamental e identificação de limitações:

01.01 – Estado inicial do menu principal com **5 opções** (sem função de exportação), mostrando a estrutura básica da interface.

01.02 – Demonstração do fluxo básico: adição do material "Ferro" (100 unidades), consulta individual e exibição do estado geral do stock.

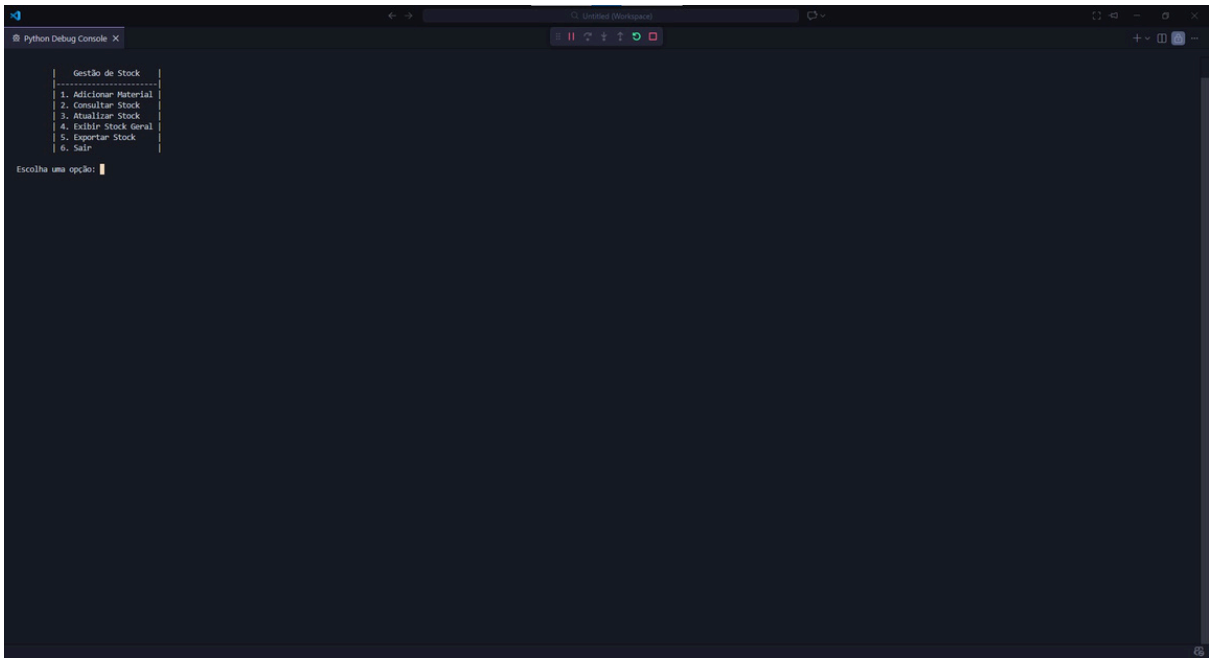
01.03 – **Teste de validação fraca:** Tentativa de adicionar "Ouro" com quantidade negativa (-10), que o sistema aceita indevidamente, revelando a necessidade de melhorias nas funções de validação.

01.04 – Continuação do teste de recuperação (atualização de Ouro para 0) e testes de robustez: consulta de material inexistente ("alien") e tentativa de atualizar material não registado ("icognita").

01.05 – Encerramento do programa através da opção 5 (Sair), finalizando a sessão da versão original.

3.2 Fase 2: Versão Melhorada (6 opções)

Python Debug Console 01.01-05



```
Python Debug Console

        |    Gestão de Stock    |

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 1
Quantidade inicial de Cobre: 100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 1
Nome do material: cobre
Quantidade inicial de Cobre: 100
Cobre adicionado com sucesso!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 2
Nome do material para consulta: ouro
Ouro não encontrado no stock.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 2
Nome do material para consulta: cobre
O stock atual de Cobre é: 100
```

```
Python Debug Console

Escolha uma opção: 2
Nome do material para consulta: cobre
O stock atual de Cobre é: 100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
------------------------------
Cobre           100

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 3
Nome do material a atualizar: cobre
Deseja adicionar (A) ou remover (R)? a
Quantidade: -100
Erro: Quantidade não pode ser negativa!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 3
Nome do material a atualizar: cobre
Deseja adicionar (A) ou remover (R)? a
Quantidade: 10
10 unidade(s) adicionada(s) ao stock de Cobre.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
```

```
Python Debug Console

        | 6. Sair              |

Escolha uma opção: 3
Nome do material a atualizar: cobre
Deseja adicionar (A) ou remover (R)? a
Quantidade: 10
10 unidade(s) adicionada(s) ao stock de Cobre.

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
-------------------------------
Cobre           110

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 1
Nome do material: rodrigo
Quantidade inicial de Rodrigo: 1
Rodrigo adicionado com sucesso!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
-------------------------------
Cobre           110
Rodrigo         1
```

```
Python Debug Console

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 1
Nome do material: rodrigo
Quantidade inicial de Rodrigo: 1
Rodrigo adicionado com sucesso!

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 4

Estado Geral do Stock:
Material        Quantidade
-------------------------------
Cobre           110
Rodrigo         1

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 5
Stock exportado com sucesso para: 2026.03.17.09.25.31-Stock_Export.txt

        |    Gestão de Stock    |
        |-----------------------|
        | 1. Adicionar Material |
        | 2. Consultar Stock    |
        | 3. Atualizar Stock    |
        | 4. Exibir Stock Geral |
        | 5. Exportar Stock     |
        | 6. Sair               |

Escolha uma opção: 6
Encerrando o programa...
PS K:\Anunali>
```

Estas capturas documentam a versão final após a implementação de funções adicionais e validações robustas:

02.01 – Menu reestruturado com **6 opções**, onde a opção 5 passa a ser "Exportar Stock" e a opção 6 "Sair", evidenciando a expansão funcional.

02.02 – Testes iniciais da nova versão: adição de "Cobre" (100 unidades), consulta de material inexistente ("ouro") que retorna mensagem de erro apropriada, e consulta bem-sucedida do cobre.

02.03 – **Demonstração de validação robusta:** Tentativa de adicionar quantidade negativa (-100) ao cobre, que agora é rejeitada com mensagem "Erro: Quantidade não pode ser negativa!", provando o funcionamento das melhorias nas funções.

02.04 – Fluxo completo de atualização (adição de 10 unidades ao cobre, totalizando 110), adição de novo material ("Rodrigo", 1 unidade) e visualização do stock atualizado com múltiplos materiais.

02.05 – **Demonstração da nova funcionalidade:** Utilização da opção 5 para exportar o stock (gerando ficheiro com timestamp 2026.03.17.09.25.31-Stock_Export.txt) e encerramento correto via opção 6.

3.3 Mapa de Progressão Funcional

Sequência	Versão	Objetivo da Demonstração
01.01 → 01.05	**Original (5 ops)**	**Fluxo básico e identificação de falhas (valores negativos aceites)**
02.01 → 02.05	**Melhorada (6 ops)**	**Validações implementadas e persistência de dados (exportação)**

Nota para o contexto académico: A numeração (01 vs. 02) indica **iterações do projeto**, não apenas ângulos diferentes. As fotos da série 02 devem ser apresentadas como evidência da capacidade das funções em suportar **melhorias incrementais** sem reescrever o programa inteiro, ilustrando o princípio da modularidade em programação estruturada.

4. Evolução do Programa: Original vs. Melhorado

Analisando as evidências documentadas nas secções 3.1 e 3.2, identificam-se melhorias cruciais implementadas através da estrutura por funções:

Aspeto	Versão Original	Versão Melhorada (Entregue)
Validação de Entrada	Conversão direta de int(input()) causava erros se o utilizador inserisse texto	Utilização de try-except para capturar ValueError e mensagens amigáveis
Quantidades Negativas	Aceitava valores negativos (lógica incoerente)	Verificação if quantidade < 0 com retorno imediato da função
Persistência de Dados	Não existia	Nova função file_export() gera ficheiros com timestamp (ex: 2026.03.17.09.25.31-Stock_Export.txt)
Feedback ao Utilizador	Mensagens básicas	Mensagens contextuais (ex: "Cobre adicionado com sucesso!", "Erro: Quantidade não pode ser negativa!")

5. Sugestões de Melhoria Futuras (Nível 10.º Ano)

Para evoluções posteriores do projeto, recomenda-se:

Funções de Validação Reutilizáveis: Criar funções auxiliares como validar_numero_positivo() ou material_existe() para evitar repetição de código nas funções adicionar_material e atualizar_stock.

Constantes para o Menu: Definir constantes (ex: OPCAO_ADICIONAR = "1") para tornar o código da função main() mais robusto e legível.

Documentação Docstring: Adicionar docstrings às funções para explicar parâmetros (ex: :param stock: Dicionário com materiais e quantidades), prática profissional importante para documentação técnica.

Tratamento de Case-Insensitive: Embora .capitalize() seja usado, uma função normalizadora de input garantiria consistência total (ex: "cobre", "COBRE" e "Cobre" serem tratados igualmente).

Função de Importação: Complementar file_export() com uma função importar_stock() para carregar dados previamente guardados, demonstrando manipulação completa de ficheiros.

6. Conclusão

As funções transformaram um script linear numa aplicação modular e robusta. A estrutura permite adicionar funcionalidades (como a exportação demonstrada na fotografia 02.05) sem "quebrar" o código existente.

Para o contexto académico do 10.º ano, este exemplo ilustra perfeitamente como a programação estruturada facilita o desenvolvimento colaborativo, a depuração de erros (visível na transição da versão 01.03 para a 02.03, onde se passou a aceitar apenas valores válidos) e a expansão gradual de projetos de software. A numeração sequencial das capturas de ecrã serve como prova tangível de que um programa bem organizado em funções permite evoluções seguras e documentadas ao longo do seu ciclo de vida.

Os Fundos Europeus mais próximos de si.